Favorite MP? so far....

clicker.cs.illinois.edu

Q1

~Code~

340

CS 340

Internet Big Picture and APIs

Updates

1. MP6 - If the files you submitted before the deadline work locally and your regrade was denied, show **Prof. Schatz** in office hours before **April 15th** for full credit back.
2. MP 8 - aiohttp lite due today
3. MP 9 - weather filter due next week ON YOUR VM
4. HW 7 due Wednesday
5. CA applications out, deadline April 27th

Internet Big Picture and APIs

Today's LGs - Build up your mental model of how the internet, IP, TCP, UDP, client-sever, API all fit together.

Be able to answers questions about what parts of a internet packet do.

Be able to describe the trade-off between TCP and UDP

Be familiar with the idea of webSockets

Be able to write and use an API

Agenda

1. Internet Big Picture
    a. TCP vs. UDP
    b. Communication Models
        i. Client-Server
        ii. Peer-to-peer
        iii. webSockets
2. webAPI

What does the IP layer provide?

A. Information of what computer the packet needs to go to.

B. Information of what port the packet needs to go to.

C. Information of what process the packet needs to go to.

D. Information of how the payload in the packet is formatted.

clicker.cs.illinois.edu

Q2

~Code~

340

What does the OS provide?

clicker.cs.illinois.edu

Q3

A. Information of what computer the packet needs to go to.

B. Information of what port the packet needs to go to.

C. Information of what process the packet needs to go to.

D. Information of how the payload in the packet is formatted.

What does the TCP/UDP layer provide?

clicker.cs.illinois.edu

Q4

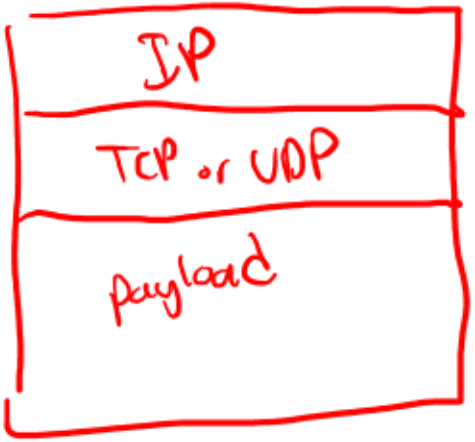
~Code~
340

A. Information of what computer the packet needs to go to.

B. Information of what port the packet needs to go to.

C. Information of what process the packet needs to go to.

D. Information of how the payload in the packet is formatted.



What does the application layer provide?

HTTP
DNS

A. Information of what computer the packet needs to go to.

B. Information of what port the packet needs to go to.

C. Information of what process the packet needs to go to.

D. Information of how the payload in the packet is formatted.

clicker.cs.illinois.edu

Q5

~Code~
340

The Internet

A global system of computers that send packets of information

Reliability Vs. Speed Tradeoff

TCP - guarantees arrival and order of packets

- more setup
- bigger headers

] slower

UDP - guarantees nothing

- faster
- less overhead

- great for small things
- great for things where speed > completeness

Example: Multiplayer Racing Game

When building a racing game, which should use TCP? The options not selected would use UDP.

clicker.cs.illinois.edu

Q6

~Code~
340

A. Packets relaying information of where certain players are (player position) UDP

B. Packets relaying information of if a player got a power up TCP

C. Packets relaying information of when a player passes the finish line TCP

Internet Communication Models

- Client-Server
- Peer-to-peer
- Mix

web sockets

Client-Server

Server - a role a process takes

- listen and respond

hardware running a process with this role

The process with this role

Client - a role a process takes

- send requests and get responses ← initiates

Client-Server Example: HTTP

Protocol

Uses TCP

defines

- requests — from client
- responses — from server

Does HTTP use TCP or UDP?

application

A. TCP - it has to for the client-server model

B. TCP - HTTP assumes reliability to work

C. UDP - HTTP doesn't assume reliability and prefers speed

D. UDP - is can't use TCP for the client-server model

clicker.cs.illinois.edu

Q7

~Code~

340

Client-Server Example: DNS

Do you think DNS uses TCP or UDP?

clicker.cs.illinois.edu

Q8

~Code~
340

A. TCP - it has to for the client-server model

B. TCP - DNS assumes reliability to work

C. UDP - DNS is small enough to fit in a packet and prefers speed

D. UDP - is can't use TCP for the client-server model

Internet Communication Models

- Client-Server
- Peer-to-peer
- Mix

Mix: WebSockets

application level protocol

like HTTP or DNS

- persistent two-way connection

AIOHTTP – upgrade the HTTP connection to a websocket connection

- great for games, live chat google docs.

API – application programming interface

defined way for 2 programs to talk to each other

Web API – API accessed over the internet

usually use client-server and HTTP

Defines

- what you can ask for
- How to ask for it
- what you will get back

Web API Summary

~~Defines~~

implemented as a server
described in documentation

gives back. json of formatted data
a program can use

API Example - We can make one with AIOHTTP

```
from aiohttp import web
import random

routes = web.RouteTableDef()

facts = [
    "The sky is blue",
    "The grass is green",
]

@routes.get("/fact")
async def get_fact(request: web.Request) -> web.Response:
    fact = random.choice(facts)
    return web.Response(text=fact)
```

Send get requests to /fact

Working with an API

A) Use curl commands to access API endpoints

B) Use code to access API endpoints

get /fact

API Example - Accessing the API endpoint

```
facts = [
    "The sky is blue",
    "The grass is green",
]

@routes.get("/fact")
async def get_fact(request: web.Request) -> web.Response:
    fact = random.choice(facts)
    return web.Response(text=fact)
```

```
curl -X GET http://127.0.0.1:8080/fact
```

API Example - Accessing the API endpoint

```python
@routes.post("/fact")
async def add_fact(request: web.Request) -> web.Response:
    data = await request.json()
    fact = data["fact"]

    facts.append(fact)

    return web.json_response({
        "message": "Fact added!",
        "fact": fact
    }, status=201)
```

```
curl -X POST http://127.0.0.1:8080/fact -H "Content-Type: application/json" -d '{"fact": "Cats sleep 16 hours a day"}'
```

API Example - Accessing the API endpoint

```
@routes.get("/fact")
async def get_fact(request: web.Request) -> web.Response:
    index = int(request.query.get("index", 0))
    fact = facts[index % len(facts)]

    return web.json_response({"fact": fact})
```

key value

curl -X GET "http://127.0.0.1:8080/fact?index=1"

after ? in the url

seperated by &

.../path?key1=val&key2=val&key3=val

API Example - Accessing the API endpoint

server

user ~~API~~

```python
from aiohttp import web, ClientSession

routes = web.RouteTableDef()

@routes.get("/get-fact-from-api")
async def get_fact_from_api(request: web.Request) -> web.Response:
    async with ClientSession() as session:
        async with session.get("http://127.0.0.1:8080/fact") as resp:
            data = await resp.json()

    return web.json_response({
        "source": "external API",
        "data": data
    })
```

fact API

```python
@routes.post("/fact")
async def add_fact(request: web.Request) -> web.Response:
    data = await request.json()
    fact = data["fact"]

    facts.append(fact)

    return web.json_response({
        "message": "Fact added!",
        "fact": fact
    }, status=201)
```

client

API Example - Art Institute of Chicago API

ART INSTITUTE CHICAGO

Art Institute of Chicago API

openapi.json

GitHub

Documentation

- Introduction
  - Quick Start
  - Conventions
  - Best Practices
- Data Dumps
  - Data Dumps vs. API?
  - Scraping Data
- Images
  - IIIF Image API
  - Image Sizes
  - IIIF Manifests
  - Image Dumps
  - Scraping Images
  - Copyright
- General Information

INTRODUCTION

The Art Institute of Chicago 's API provides JSON-formatted data as a REST-style service that allows developers to explore and integrate the museum's public data into their projects. This API is the same tool that powers our website , our mobile app , and many other technologies in the museum.

If the application you're building will be public, please send it our way! We'd love to share it alongside some of the other projects that use our API . And if you have any questions, please feel free to reach out to us: engineering@artic.edu.

API Example - Art Institute of Chicago API

`GET /artworks`

A list of all artworks sorted by last updated date in descending order. For a description of all the fields included with this response, see here.

AVAILABLE PARAMETERS:

- `ids` - A comma-separated list of resource ids to retrieve
- `limit` - The number of resources to return per page
- `page` - The page of resources to retrieve
- `fields` - A comma-separated list of fields to return per resource
- `include` - A comma-separated list of subresource to embed in the returned resources. Available options are:
  - `artist_pivots`
  - `dates`
  - `place_pivots`
  - `sites`

API Example - Art Institute of Chicago API

GET

```
% curl "https://api.artic.edu/api/v1/artworks?limit=1&fields=title,id"
":1,"offset":0,"total_pages":131565,"current_page":1,"next_url":"https:\/\/api.
limit=1&fields=title%2Cid"},"data":[{"id":28067,"title":"The Old Guitarist"}],"
ion` field in this response is licensed under a Creative Commons Attribution 4.
rms and Conditions of artic.edu. All other data in this response is licensed un
0 designation and the Terms and Conditions of artic.edu.","license_links":["htt
main\/zero\/1.0\/","https:\/\/www.artic.edu\/terms"],"version":"1.14"},"config"
u\/iiif\/2","website_url":"http:\/\/www.artic.edu"}}
```

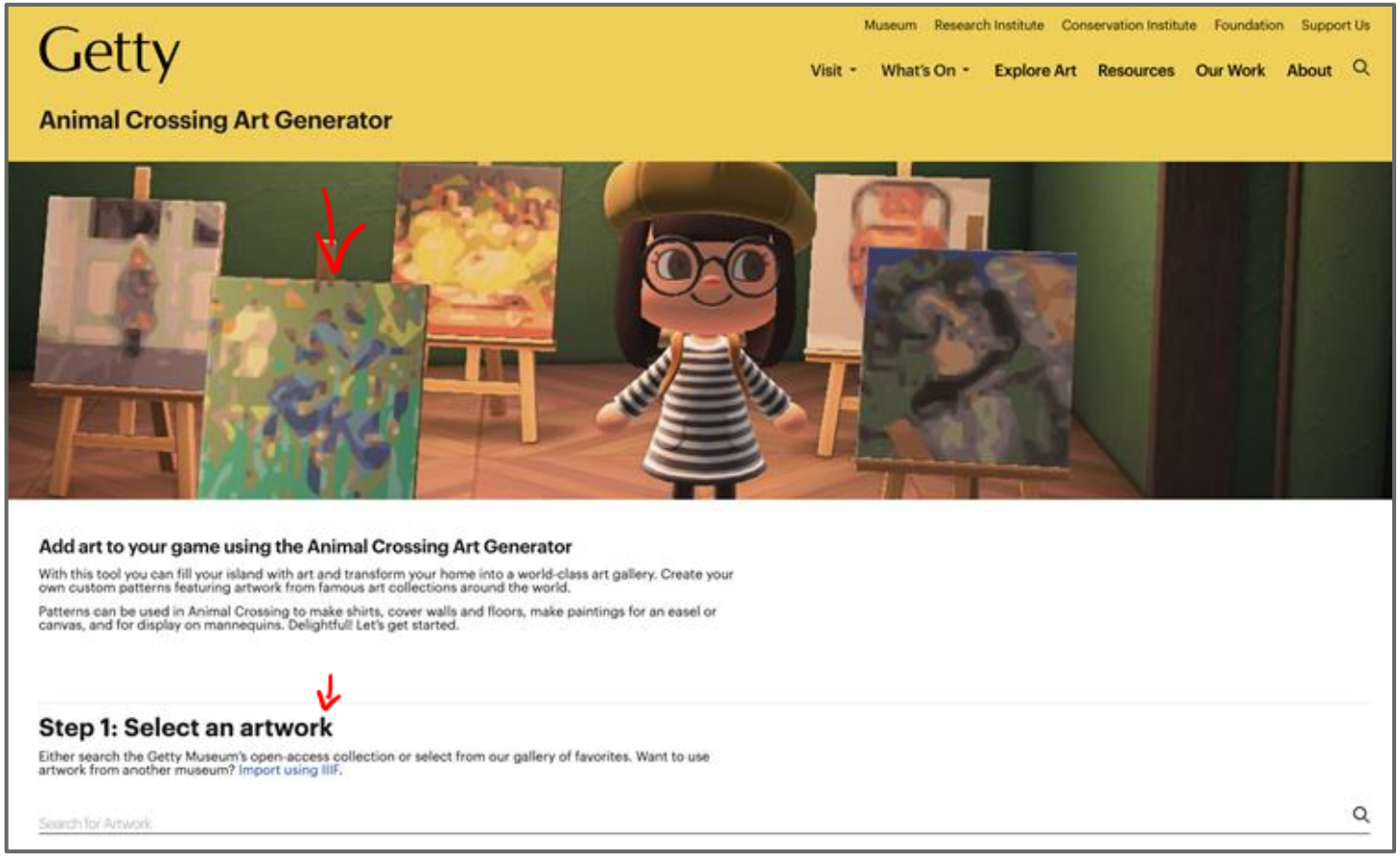
API Example - Art Institute of Chicago API

```
@routes.get("/monet")
async def get_monet(request: web.Request) -> web.Response:
    async with ClientSession() as session:
        async with session.get(
            "https://api.artic.edu/api/v1/artworks/search",
            params={
                "q": "Monet",
                "limit": 5
            }
        ) as resp:
            data = await resp.json()

    artworks = [art["title"] for art in data["data"]]

    return web.json_response({
        "artist": "Monet",
        "artworks": artworks
    })
```

API Example - Art Institute of Chicago API



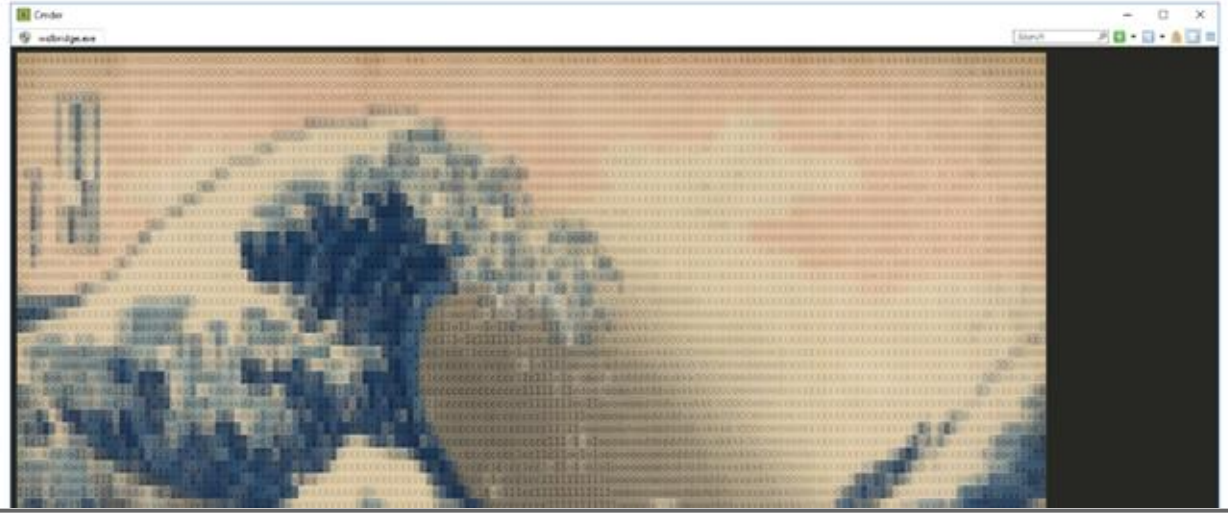
API Example - Art Institute of Chicago API

aic-bash

> A bash script to query our API for public domain artworks and render them as ASCII art

Just a small side-project we did to show what could be done with our museum's API.

The Art Institute of Chicago (AIC) offers all of its public data via a centralized, searchable API. There, you can find data on artworks, artists, and places represented in our collection, and much more besides. For more information, check out our Open Access initiatives.



API – server interface

Server – a running process that listens for requests

Process – a program running on a computer

Python - variable number of arguments

```
def add(*args):
    print(*args)
```

add * to unpack

(1, 2, 5, 6)

```
add(1, 2, 5, 6)
```

```
def print_info(**kwargs)
    print(kwargs)
```

dictionary

**

```
print_info(val=4, size_=6)
```